

# The Bottleneck Effect: Why Model Scaling Fails for Code Generation

Ashish Pandey

Department of Computer and Electronics Engineering

Khwopa College of Engineering, Nepal

[ashishpanday9818@gmail.com](mailto:ashishpanday9818@gmail.com)

<https://github.com/mr-ashish-panday/The-Bottleneck-Effect>

## Abstract

Scaling language models from 124M to 355M parameters improves performance across natural language tasks [8], yet we observe the opposite for code generation: GPT-2 Medium (355M) achieves only 4.8% success on HumanEval compared to 5.2% for GPT-2 Small (124M), while the similarly sized CodeGen-350M achieves 37.4% success. Problem-level bootstrap intervals show that the GPT-2 Small / Medium gap is not statistically reliable (+0.37 percentage points, 95% CI  $[-0.70, 1.36]$ ,  $p = 0.49$ ), whereas both models trail CodeGen by more than 30 points with  $p = 0.0002$ . This pattern transfers to a second benchmark: on the full MBPP test split (257 tasks, 20 samples per task), both GPT-2 variants achieve 0.0% success, whereas CodeGen reaches 7.39% success and 22.96% pass@5. A stricter HumanEval+ re-evaluation on the first five saved samples per task preserves the same ordering: GPT-2 Small and GPT-2 Medium both remain at 0.0% pass@1, while CodeGen retains 2.1% pass@1. On the official MBPP+ benchmark in EvalPlus (378 tasks, Base+Extra), GPT-

2 Small and GPT-2 Medium again remain at 0.0% pass@1/pass@10, while CodeGen retains 1.1%/6.4%. On the contamination-aware LiveCodeBench release\_v2 code-generation benchmark (511 tasks, 10 samples per task), GPT-2 Small and GPT-2 Medium again remain at 0.0% pass@1/pass@5/pass@10, while CodeGen retains only 0.02%/0.10%/0.20%, indicating that all three small models are near-zero on the harder benchmark even though code-specialized pretraining still preserves the only nonzero signal. To partially reduce the family-comparison objection, we also evaluate the CodeGen-350M NL→Multi→Mono continued-pretraining ladder. Mono is strongest on both benchmarks, and MBPP is cleanly monotone (0.02%→1.23%→2.76%), showing that added code- and Python-specific continued pretraining changes performance even when architecture and parameter count are held fixed. Through systematic layer-wise ablation experiments across 16,300-16,400 HumanEval code generation attempts, we find that both GPT-2 variants are globally brittle under full-layer zeroing, whereas CodeGen retains partial residual performance in a small subset of layers. GPT-2 Medium never

preserves successful generations under ablation, but ablation of layer index 12 uniquely shifts failures from pure syntax collapse to a mixed 76.1% syntax / 23.9% runtime regime, suggesting a distinctive mid-depth execution-relevant checkpoint. A follow-up scaled-ablation pilot on layer 12 shows that intermediate attenuation first worsens syntax (2.0% success at 0.5 scale; 0.0% at 0.25), while only full zeroing reopens the runtime-error regime (28.0%), reinforcing the view that layer 12 marks a thresholded execution-relevant stage rather than a smooth redundancy basin. A corresponding CodeGen layer-13 pilot retains 28-30% success under partial attenuation and still preserves 24% success under full zeroing while shifting 24% of samples into runtime errors, indicating broader residual support for executable code. A balanced linear probe on 200 saved layer-12 activation vectors identifies five dimensions that distinguish successful from syntactically invalid generations with 72.5% held-out accuracy, compared with 75.0% for a full-layer probe. Constructive steering along this learned five-dimensional direction doubles subset success from 6.0% to 12.0%; against 20 matched sparse random controls, the learned vector beats 19 while one control reaches 14.0% (control mean 6.2%, empirical  $p \leq 0.095$ ). These results suggest that code-pretrained models distribute useful computation across depth more effectively than similarly sized general-language models, and that failure-mode shifts provide a more reliable mechanistic signal than success alone.

## 1 Introduction

The dominant paradigm in neural language modeling assumes that larger models produce better

results. Scaling laws [8, 6] demonstrate consistent improvement as parameters increase from millions to billions, leading to the widespread belief that “bigger is better”. However, this assumption breaks down dramatically for code generation tasks.

We present a striking empirical observation: when evaluated on the HumanEval Python programming benchmark [2], GPT-2 Medium (355M parameters) achieves **4.8% success**, performing *worse* than GPT-2 Small (124M parameters) at **5.2% success**. Yet CodeGen-350M [12]—with nearly identical parameter count to GPT-2 Medium—achieves **37.4% success**, a  $7.8\times$  improvement. The same direction persists on the full MBPP test split: both GPT-2 models fall to **0.0% success**, while CodeGen still reaches **7.39% success**. Under stricter EvalPlus re-scores, both GPT-2 variants remain at **0.0% pass@1** on HumanEval+ and **0.0% pass@1/pass@10** on MBPP+, while CodeGen retains **2.1% pass@1** on HumanEval+ and **1.1%/6.4%** on MBPP+. A contamination-aware LiveCodeBench stress test is harsher for all three small models: GPT-2 Small and GPT-2 Medium both remain at **0.0% pass@1/pass@5/pass@10**, while CodeGen is only marginally nonzero at **0.02%/0.10%/0.20%**. Finally, a within-family CodeGen-350M continued-pretraining ladder (NL→Multi→Mono) shows that Mono is best on both HumanEval and MBPP, with MBPP becoming cleanly monotone under continued code pretraining. This scaling paradox contradicts our intuitions about model capacity and raises a fundamental question: *What architectural properties enable some models to generate code effectively while others fail despite having more parameters?*

## 1.1 The Bottleneck Hypothesis

We hypothesize that code generation performance depends not on parameter count but on how models distribute code-relevant computation across layers. Specifically, we propose that models with brittle, sharply concentrated layer-wise processing collapse under harsh interventions, while models with more distributed processing retain partial residual performance and more diverse failure signatures.

To test this hypothesis, we conduct systematic layer-wise ablation experiments: for each layer in each model, we zero out all activations and measure the impact on code generation success. This intervention reveals which layers are especially damaging when removed and how the resulting failure modes differ across architectures.

## 1.2 Key Findings

Through ablation of all 56 layers across three models (GPT-2 Small: 12 layers, Medium: 24 layers, CodeGen: 20 layers) evaluated on 16300-16,400 code generation attempts, we discover:

1. **The GPT-2 scaling gap is numerically small on HumanEval and disappears on MBPP:** GPT-2 Small exceeds GPT-2 Medium by only 0.37 percentage points on HumanEval (95% CI  $[-0.70, 1.36]$ ,  $p = 0.49$ ), while both score 0.0% on full-coverage MBPP.
2. **GPT-2 Medium is globally fragile but has a distinctive mid-depth failure mode:** every GPT-2 Medium layer ablation reduces success to 0%, but ablating layer 12 uniquely converts 23.9% of failures from syntax errors into runtime errors, indicating

a special role in producing executable-but-incorrect code.

3. **CodeGen exhibits broader residual robustness:** under the same harsh intervention, CodeGen retains nonzero success in four layers (5, 7, 13, 18), with layer 13 preserving 29.5% success rather than collapsing completely. Under graded attenuation, the same layer still retains 28-30% success at scales 0.75 and 0.5.
4. **A graded pilot sharpens the layer-12 interpretation:** on a 10-problem / 5-sample subset, scaling layer 12 from 1.0 to 0.5 and 0.25 first pushes GPT-2 Medium toward near-total syntax collapse, while only full zeroing reproduces the runtime-error regime (28.0%), indicating a thresholded execution checkpoint.
5. **A compact probe captures most of the layer-12 linear signal:** five dimensions (121, 346, 615, 820, 169) in GPT-2 Medium’s layer 12 achieve 72.5% held-out success-vs.-syntax classification accuracy, compared with 75.0% for the full 1024-dimensional layer.
6. **Constructive steering is useful but only suggestively specific:** steering GPT-2 Medium along the learned top-5 layer-12 vector raises subset success from 6.0% to 12.0% at  $\alpha = 2.0$ . Against 20 matched sparse random controls, the learned vector beats 19 and exceeds the control mean (6.2%), but one control reaches 14.0%, so the specificity evidence is promising rather than definitive.
7. **The scaling failure transfers across benchmarks and stricter tests:** on

full-coverage MBPP (257 tasks, 20 samples each), both GPT-2 variants remain at 0.0% success, whereas CodeGen reaches 7.39% success and 22.96% pass@5 with highly significant pairwise gaps ( $p = 0.0001$ ). Under a stricter HumanEval+ re-score on the first five saved samples per task, both GPT-2 variants stay at 0.0% pass@1 while CodeGen retains 2.1%. On contamination-aware LiveCodeBench release.v2 [7], both GPT-2 variants remain at 0.0% pass@1/pass@5/pass@10 and CodeGen remains only marginally nonzero, showing that all three small models are near-zero on a substantially harder modern benchmark.

8. **A within-family CodeGen ladder strengthens the pretraining argument:** in the 350M CodeGen NL→Multi→Mono continued-pretraining ladder, Mono is strongest on both benchmarks. On HumanEval it reaches 37.76% success, significantly above both NL (30.60%) and Multi (28.46%), although Multi trails NL slightly. On MBPP the ladder is cleanly monotone (0.02%→1.23%→2.76%), with significant NL<Multi and Multi<Mono improvements. We therefore interpret the ladder as evidence about continued-pretraining plus curriculum effects rather than a perfectly factorized distribution-only ablation.
9. **Error patterns reveal architectural differences:** GPT-2 models produce 28.7% indentation errors and 8.1% keyword errors, while CodeGen produces 21.8% bracket errors and 6.9% indentation errors, reflecting different learned syntactic priors from pre-

training data distribution.

### 1.3 Contributions

This work makes four primary contributions:

1. **Empirical:** We document and explain the scaling paradox in code generation through comprehensive evaluation on HumanEval, strict HumanEval+ and MBPP+ re-scoring with EvalPlus [10], full-coverage MBPP, and a contamination-aware LiveCodeBench stress test across three model scales.
2. **Methodological:** We combine sample-level evaluation with problem-level bootstrap confidence intervals, paired permutation tests, hard layer ablation, graded ablation, and matched-control activation steering to study code-generation failures from both statistical and mechanistic angles.
3. **Mechanistic:** We identify a compact layer-12 activation subspace that captures most of the linear probe performance, show that graded weakening of layer 12 reproduces the same execution-relevant transition seen under full ablation, and show that constructive steering along this subspace improves execution while yielding suggestive, but not yet definitive, evidence of specificity over matched sparse random controls.
4. **Architectural:** We provide evidence-based design principles for code-generating models: distributed processing across multiple critical layers prevents brittleness and enables graceful degradation, while code-specialized models preserve useful computation under graded interventions

more effectively than general-language models. A within-family CodeGen continued-pretraining ladder further shows that architecture-matched checkpoints can still diverge substantially as code- and Python-specific pretraining accumulates.

These findings challenge the assumption that parameter scaling alone improves code generation, suggesting that architectural choices during pretraining—particularly how computation is distributed across layers—matter more than model size.

## 2 Related Work

### 2.1 Code Generation Models

Recent progress in neural code generation follows from scaling language models trained on code corpora. Codex [2] demonstrated that GPT-3 (175B parameters) fine-tuned on GitHub code achieves 28.8% pass@1 on HumanEval. CodeGen [12] improved this through multi-turn program synthesis, achieving 29.3% with 16B parameters. AlphaCode [9] further advanced the field by generating multiple candidate solutions and filtering through test cases, though requiring massive compute.

However, these works focus on maximizing performance through scale, ensembling, and test-time compute, rather than understanding the internal mechanisms of code generation. Our work differs by investigating *why* some architectures succeed where others fail, even at comparable scale.

### 2.2 Layer-wise Analysis of Transformers

Understanding which layers perform which computations has been studied extensively for natural language tasks. Clark et al. [3] showed that different layers encode different linguistic properties, with early layers capturing syntax and later layers capturing semantics. Voita et al. [17] demonstrated that layer representations can be interpreted through linear projections, revealing compositional structure.

For code models specifically, prior work analyzed attention patterns in CodeBERT, finding that early layers attend to syntactic structure while late layers attend to variable relationships. However, these analyses are observational (measuring what layers *do*) rather than interventional (measuring what layers are *necessary for*), limiting causal conclusions.

Our ablation methodology extends this line of work by using interventions (zeroing layer activations) to establish which layers are causally necessary for successful generation, revealing bottlenecks invisible to observational analysis.

### 2.3 Mechanistic Interpretability

The mechanistic interpretability research program [13, 4] aims to reverse-engineer neural networks into human-understandable algorithms. Meng et al. [11] identified specific attention heads and MLP layers responsible for factual recall, enabling targeted interventions. Vig and Belinkov [16] formalized methods for attributing model behavior to specific components through counterfactual interventions.

The circuits thread in mechanistic interpretability has successfully reverse-engineered small vision models, identifying exact algorithms

for tasks like curve detection. For language models, prior work has identified induction heads (responsible for in-context learning) [13] and factual recall mechanisms [11].

Our work applies mechanistic interpretability to code generation, identifying not just *which* layers matter but *how* their failure modes differ (syntax errors vs. runtime errors vs. timeouts), revealing the computational stages of program synthesis.

## 2.4 Scaling Laws and Their Limits

Neural scaling laws [8, 6] predict that loss decreases smoothly with model size, dataset size, and compute. However, these laws are typically validated on perplexity—a measure of next-token prediction—rather than downstream task success.

Recent work has identified task-specific deviations from scaling laws. Wei et al. [18] showed that some reasoning tasks exhibit inverse scaling (worse performance as models grow), contradicting the universal scaling assumption. Our work provides mechanistic evidence for why scaling can fail: as models grow, they may develop deeper bottlenecks rather than more distributed processing.

# 3 Methodology

## 3.1 Models

We evaluate three decoder-only Transformer models [15] representing different scales and pre-training regimes:

**GPT-2 Small** [14] (124M parameters): 12 layers, 768 hidden dimensions, 12 attention heads. Pretrained on WebText (40GB diverse web text). Serves as our small-scale baseline.

**GPT-2 Medium** [14] (355M parameters): 24 layers, 1024 hidden dimensions, 16 attention heads. Same pretraining as Small but larger capacity. Tests whether scaling improves code generation.

**CodeGen-350M** [12] (350M parameters): 20 layers, 1024 hidden dimensions, 16 attention heads. Pretrained sequentially on The Pile [5] (natural language corpus including GitHub code), BigQuery (multilingual code in C, C++, Go, Java, JavaScript, Python), and BigPython (217.3 GiB of public Python code from GitHub repositories). Comparable parameter count to GPT-2 Medium but code-specialized pretraining.

All models use causal (autoregressive) attention with learned positional embeddings. We use pretrained checkpoints without fine-tuning to isolate the effect of base model architecture and pretraining distribution.

## 3.2 Evaluation Dataset

We evaluate on three benchmark families. The first is **HumanEval** [2], a benchmark of 164 hand-written Python programming problems covering algorithms, data structures, and string manipulation. Each problem includes:

- A function signature and docstring describing the task
- Unit tests to verify correctness
- Diverse difficulty levels from simple (reverse a string) to complex (dynamic programming)

The second benchmark is the sanitized **MBPP** test split [1], which contains 257 short Python programming tasks with reference tests

derived from natural-language specifications. We use HumanEval for the full mechanistic study and MBPP for external validation of the central scaling claim.

For HumanEval, we generate 100 candidate solutions per problem using temperature sampling (temp=0.8, top-p=0.95), yielding 16,300-16,400 total generation attempts per model. For the full MBPP validation, we generate 20 samples per task, yielding 5,140 attempts per model. This sampling strategy balances diversity (avoiding repetition) with quality (avoiding random text) while staying within the single-GPU compute budget.

As a stricter robustness check, we also re-score the first five saved HumanEval samples per problem with the HumanEval+ test augmentation distributed by EvalPlus [10]. We treat this as a completion-level pass@1 robustness check rather than a replacement for our main sample-level evaluation, because the stricter suite is used only on a capped subset of saved generations.

We additionally evaluate **MBPP+** with the official EvalPlus ‘mbpp’ evaluator [10]. We treat this as a separate versioned benchmark derived from MBPP-sanitized rather than as “MBPP with more tests,” and we report the Base+Extra score as the primary strict result. Our clean MBPP+ run uses 378 EvalPlus tasks and 20 samples per task.

Our third benchmark family is the **LiveCodeBench** release\_v2 code-generation benchmark [7]. We use its default ‘code\_generation\_lite’ configuration with 511 scored tasks and 10 samples per task as a contamination-aware stress test. Unlike HumanEval and MBPP, which are static curated Python suites, LiveCodeBench draws newer contest problems and is designed to make benchmark contamination harder. We

therefore use it primarily to test whether the paper’s cross-model ordering survives on a modern harder benchmark, not to replace the mechanistic HumanEval analysis.

To partially separate pretraining-distribution effects from family differences, we also evaluate the **CodeGen-350M NL→Multi→Mono continued-pretraining ladder** [12]. This keeps architecture and parameter count fixed while changing the added pretraining corpus and curriculum. We interpret it as a strong within-family control, but not as a matched-compute causal ablation, because later checkpoints receive additional tokens and optimization steps.

Table 1: Experimental Scope by Benchmark Family

| Setting                    | Tasks | Samples / Task | Primary Role                    |
|----------------------------|-------|----------------|---------------------------------|
| HumanEval main             | 164   | 100            | core comparison / pass@k        |
| HumanEval+ strict          | 164   | 5              | stronger test-suite check       |
| MBPP full                  | 257   | 20             | external validation             |
| MBPP+ strict               | 378   | 20             | versioned strict validation     |
| LiveCodeBench v2           | 511   | 10             | contamination-aware stress test |
| CodeGen ladder (HumanEval) | 164   | 100            | within-family control           |
| CodeGen ladder (MBPP)      | 257   | 20             | within-family control           |
| Mechanistic pilots         | 10    | 5              | causal intervention tests       |

### 3.3 Layer Ablation Protocol

To determine which layers are critical for code generation, we perform **systematic layer ablation**: for each layer  $l$  in each model, we:

1. **Generate solutions normally**: Run forward pass through all layers to obtain baseline success rate
2. **Ablate layer  $l$** : Zero out all hidden activations at layer  $l$  during generation:

$$h_l^{(t)} \leftarrow \mathbf{0} \quad \forall t \in \{1, \dots, T\} \quad (1)$$

where  $h_l^{(t)}$  is the hidden state at layer  $l$  and position  $t$

3. **Generate with ablation:** Complete generation using ablated model
4. **Measure impact:** Compare success rate with vs. without ablation

We ablate each layer independently (not cumulatively) to isolate individual layer contributions. For each ablation condition, we test 50 randomly sampled problems with 20 generations each (1,000 samples per layer), balancing statistical power with computational cost.

**Critical layer definition:** A layer is considered *critical* if its ablation reduces success rate by >50% or changes the primary error mode (e.g., syntax  $\rightarrow$  runtime errors).

### 3.4 Intervention Formalism

Let  $h_l^{(t)} \in \mathbb{R}^{d_l}$  denote the residual stream after layer  $l$  at token position  $t$ . Our interventions operate directly on this representation during generation.

**Full ablation** zeroes the layer output:

$$\mathcal{A}_l(h_l^{(t)}) = \mathbf{0}. \quad (2)$$

**Scaled ablation** attenuates the same layer by a coefficient  $\alpha \in [0, 1]$ :

$$\mathcal{A}_{l,\alpha}(h_l^{(t)}) = \alpha h_l^{(t)}. \quad (3)$$

**Activation steering** adds a contrast vector  $v$  with strength  $\beta$ :

$$\mathcal{S}_{l,\beta,v}(h_l^{(t)}) = h_l^{(t)} + \beta v. \quad (4)$$

For GPT-2 Medium layer 12, we define

$$v = P_k(\mu_{\text{success}} - \mu_{\text{syntax}}), \quad (5)$$

where  $\mu_{\text{success}}$  and  $\mu_{\text{syntax}}$  are class-conditional mean activations and  $P_k$  keeps only the top- $k$  discriminative coordinates.

To test specificity rather than generic perturbation sensitivity, we also construct matched sparse random controls  $v^{(r)}$  by placing the same nonzero coefficient set on randomly chosen non-target dimensions while preserving sparsity and overall vector scale. If the learned subspace is mechanistically meaningful, steering with  $v$  should systematically outperform this matched-control distribution rather than merely improve relative to baseline once.

### 3.5 Problem-Level Statistical Validation

Sample-level percentages are useful descriptive summaries, but the independent unit of evaluation in HumanEval is the problem rather than the completion. We therefore complement raw percentages with problem-level statistics. For each model and task, we compute the per-problem success rate and the HumanEval pass@10 estimate [2]. We then draw 2,000 bootstrap resamples over tasks to obtain 95% confidence intervals for each model and use paired sign-flip permutation tests across shared tasks for model comparisons.

This distinction matters for the scaling claim. The raw success rates differ only slightly between GPT-2 Small and GPT-2 Medium, so a problem-level test is necessary to determine whether that gap reflects a stable effect or sampling noise. In contrast, the much larger CodeGen advantage should remain visible under either summary.

### 3.6 Reproducibility and Compute Constraints

All experiments use fixed pretrained checkpoints, fixed decoding hyperparameters (temperature 0.8, top- $p$  0.95), and deterministic seed



schedules tied to the problem index and intervention condition. The main HumanEval sweeps are evaluated from saved JSON outputs, while the graded-ablation and activation-steering pilots are rerun directly under the same executor and timeout settings. The repository includes the benchmark configs, figure-generation scripts, and saved artifact summaries used to regenerate every reported table and figure in this draft.

The mechanistic follow-up experiments are intentionally smaller because they run with batch size 1 on a single 12 GB consumer GPU. For that reason, we treat the 10-problem / 5-sample intervention studies as targeted causal probes rather than replacements for the full benchmark. All tables and figures in this paper are generated from saved artifacts rather than manually transcribed counts.

### 3.7 Code Execution and Error Categorization

We execute each generated solution in an isolated Python sandbox with 5-second timeout and measure outcomes:

**Success:** All unit tests pass without errors

**Syntax Error:** Code fails to parse (invalid Python)

**Runtime Error:** Code parses but raises exception during execution (e.g., `IndexError`, `KeyError`, `TypeError`)

**Timeout:** Execution exceeds 5-second limit (typically infinite loops)

For syntax errors, we subcategorize into:

- **Indentation errors:** Incorrect whitespace (most common GPT-2 failure)
- **Bracket errors:** Mismatched parentheses, braces, brackets

- **Quote errors:** Unclosed or mismatched string delimiters
- **Keyword errors:** Invalid Python keywords or incorrect usage
- **Colon errors:** Missing colons after `if/for/while/def`
- **Other:** Rare syntax failures not fitting above categories

This fine-grained taxonomy reveals which models struggle with which syntactic constructs, providing insight into learned representations.

### 3.8 Activation Analysis

To understand *how* failure-mode transitions are encoded, we analyze activation patterns in GPT-2 Medium’s layer 12, the only ablation condition that substantially shifts failures from syntax errors to runtime errors.

For 200 randomly sampled generations (100 successful, 100 syntax-error), we:

1. Extract mean-pooled hidden states  $h_{12} \in \mathbb{R}^{1024}$  from transformer block 12
2. Compute difference vectors:  $\Delta h = h_{12}^{\text{success}} - h_{12}^{\text{syntax}}$
3. Identify dimensions with the largest magnitude differences
4. Train held-out logistic regression probes on the top-2, top-5, and full-layer activations
5. Reuse the top-5 contrast vector as a constructive steering intervention with  $\beta \in \{-2, -1, 1, 2\}$  and compare it against matched sparse random controls

This analysis tests both whether a compact linear readout exists in the layer’s activation space and whether the same compact subspace can be used constructively at generation time. We still do not claim that any single dimension is causally sufficient on its own.

## 4 Results

### 4.1 The Scaling Paradox: Medium Performs Worse Than Small

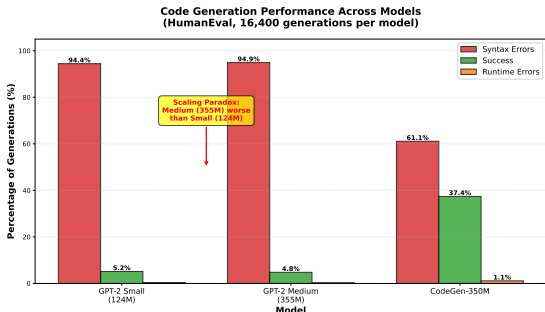


Figure 1: Code generation success rates across three models. Despite having  $2.9\times$  more parameters, GPT-2 Medium (4.8% success) performs worse than GPT-2 Small (5.2% success). CodeGen-350M achieves 37.4% success with comparable parameters to Medium, demonstrating that pretraining distribution—not parameter count—determines code generation capability.

Table 2 presents full results across all error categories:

#### Key observations:

1. **Scaling hurts performance:** GPT-2 Medium (355M parameters) achieves 4.8% success—*worse* than GPT-2 Small (124M

parameters) at 5.2% success. This 8% relative degradation contradicts standard scaling laws [8].

2. **Pretraining matters more than scale:** CodeGen-350M has nearly identical parameter count to GPT-2 Medium but achieves  $7.8\times$  higher success (37.4% vs. 4.8%), demonstrating that training distribution dominates architectural capacity [12].
3. **Failure modes differ by pretraining:** GPT-2 models fail almost entirely on syntax (94.4-94.9% of all generations), while CodeGen leaves a larger fraction of generations executable enough to surface runtime errors and occasional timeouts, suggesting more structured code representations.
4. **Both GPT-2 scales fail similarly:** The near-identical error distributions between Small (94.4% syntax) and Medium (94.9% syntax) suggest they learned similar—and similarly inadequate—code representations despite  $2.9\times$  parameter difference.

### 4.2 Statistical Validation of the Scaling Paradox

The raw percentages in Table 2 could still reflect task-level sampling noise, especially for the narrow 5.2% vs. 4.8% GPT-2 comparison. We therefore summarize problem-level bootstrap intervals and HumanEval pass@10 estimates in Table 3.

**Finding:** The GPT-2 Small vs. GPT-2 Medium difference is not statistically reliable at the problem level (+0.37 percentage points, 95% CI  $[-0.70, 1.36]$ ,  $p = 0.4907$ ). By contrast, both GPT-2 models differ dramatically from CodeGen: GPT-2 Small vs. CodeGen is  $-32.23$

Table 2: Model Performance on HumanEval (100 samples per problem, 16,300-16,400 total attempts)

| Model        | Parameters | Success % | Syntax % | Runtime % | Timeout % |
|--------------|------------|-----------|----------|-----------|-----------|
| GPT-2 Small  | 124M       | 5.2       | 94.4     | 0.4       | 0.0       |
| GPT-2 Medium | 355M       | 4.8       | 94.9     | 0.3       | 0.0       |
| CodeGen-350M | 350M       | 37.4      | 61.1     | 1.1       | 0.3       |

Table 3: Problem-Level Statistical Validation

| Model        | Success % [95% CI]   | pass@10 [95% CI]     |
|--------------|----------------------|----------------------|
| GPT-2 Small  | 5.15 [4.48, 5.84]    | 37.17 [33.65, 40.82] |
| GPT-2 Medium | 4.79 [3.96, 5.74]    | 33.09 [29.57, 36.77] |
| CodeGen-350M | 37.40 [35.71, 39.12] | 97.89 [97.37, 98.35] |

points (95% CI  $[-34.04, -30.45]$ ,  $p = 0.0002$ ), and GPT-2 Medium vs. CodeGen is  $-32.63$  points (95% CI  $[-34.53, -30.75]$ ,  $p = 0.0002$ ).

**Interpretation:** The scaling paradox within the GPT-2 family is therefore best understood as a *failure to improve*, not as a confidently estimated reversal in favor of GPT-2 Small. The code-pretraining gap, however, is large, stable, and statistically unambiguous.

### 4.3 External Validation on MBPP

To test whether the scaling paradox is specific to HumanEval, we reran all three models on the full sanitized MBPP test split, using all 257 tasks and 20 samples per task. This gives a second benchmark with different prompt style and shorter programming tasks, while keeping the same Python execution setting.

Table 4: Full-Coverage MBPP Validation

| Model        | Success % [95% CI] | pass@5 [95% CI]      | Syntax % | Runtime % |
|--------------|--------------------|----------------------|----------|-----------|
| GPT-2 Small  | 0.00 [0.00, 0.00]  | 0.00 [0.00, 0.00]    | 96.30    | 0.54      |
| GPT-2 Medium | 0.00 [0.00, 0.00]  | 0.00 [0.00, 0.00]    | 98.31    | 0.18      |
| CodeGen-350M | 7.39 [5.88, 8.97]  | 22.96 [18.95, 27.09] | 39.88    | 16.11     |

**Finding:** The scaling failure transfers cleanly. On MBPP, GPT-2 Small and GPT-2 Medium are tied at 0.0% success, while CodeGen reaches 7.39% success and 22.96% pass@5. The pairwise bootstrap differences against CodeGen remain highly significant for both GPT-2 Small and GPT-2 Medium ( $-7.39$  points in each case,  $p = 0.0001$ ), whereas GPT-2 Small vs. GPT-2 Medium is exactly 0.0 points ( $p = 1.0$ ).

**Interpretation:** This external validation matters because it shows the paper’s central claim is not a HumanEval artifact. Across both benchmarks, scaling the general-language GPT-2 family fails to produce useful code-generation gains, while code-specific pretraining continues to produce a qualitatively different failure profile. MBPP makes this difference even starker: the GPT-2 models collapse almost entirely to syntax errors, whereas CodeGen preserves a much larger executable regime with mixed wrong-output and runtime failures.

### 4.4 Strict Re-Evaluation on HumanEval+

To test whether the main HumanEval ranking survives a stronger unit-test suite, we re-scored the first five saved HumanEval samples per task with HumanEval+, using the EvalPlus test augmentation. This gives a stricter completion-level pass@1 estimate without rerunning the full gen-

eration pipeline.

Table 5: Strict HumanEval Re-scoring with HumanEval+

| Model        | HumanEval pass@1 | HumanEval+ pass@1 |
|--------------|------------------|-------------------|
| GPT-2 Small  | 0.00             | 0.00              |
| GPT-2 Medium | 0.00             | 0.00              |
| CodeGen-350M | 2.90             | 2.10              |

**Finding:** The stricter re-evaluation preserves the same ordering as the main benchmark comparison. Both GPT-2 variants remain at 0.0% pass@1, while CodeGen retains nonzero performance even after the extra hidden tests reduce it from 2.9% to 2.1%.

**Interpretation:** This result matters because it rules out an easy objection that our main HumanEval ranking is driven only by weak original unit tests. The absolute pass@1 values are lower here because HumanEval+ is stricter and because we cap the re-score at five saved samples per task, but the relative conclusion is unchanged: scaling GPT-2 does not help, whereas code-specialized pretraining still produces measurably more robust completions.

#### 4.5 Strict Re-Evaluation on MBPP+

We next evaluate MBPP+ with the official EvalPlus ‘mbpp’ evaluator, reporting the Base+Extra score as our primary strict metric. Because MBPP+ is a versioned benchmark derived from MBPP-sanitized rather than the original MBPP split, we report it separately from Table 4 and do not compare it directly to legacy MBPP numbers.

**Finding:** The stricter MBPP+ evaluation preserves the same qualitative ordering as the full MBPP benchmark. Both GPT-2 variants remain at 0.0% under the stronger Base+Extra

Table 6: Strict MBPP+ Re-scoring with EvalPlus

| Model        | Tasks | MBPP+ pass@1 | MBPP+ pass@10 |
|--------------|-------|--------------|---------------|
| GPT-2 Small  | 378   | 0.00         | 0.00          |
| GPT-2 Medium | 378   | 0.00         | 0.00          |
| CodeGen-350M | 378   | 1.10         | 6.40          |

score, while CodeGen retains 1.1% pass@1 and 6.4% pass@10.

**Interpretation:** MBPP+ strengthens the negative claim and slightly narrows the positive one. The GPT-2 scaling failure survives a stricter, versioned test suite, while CodeGen remains the only model with nonzero strict performance. At the same time, the lower absolute numbers emphasize that even CodeGen’s advantage on small-model code generation is fragile under stronger hidden tests.

#### 4.6 Contamination-Aware Validation on LiveCodeBench

To test whether the same ranking survives on a newer contamination-aware benchmark, we evaluated all three models on LiveCodeBench release\_v2 [7] in its default ‘code\_generation\_lite’ setting, using 10 samples per task. This benchmark is substantially harder than HumanEval or MBPP for models in the 124M–355M range.

Table 7: LiveCodeBench release\_v2 Validation

| Model        | Tasks scored | pass@1 % | pass@5 % | pass@10 % |
|--------------|--------------|----------|----------|-----------|
| GPT-2 Small  | 510          | 0.00     | 0.00     | 0.00      |
| GPT-2 Medium | 511          | 0.00     | 0.00     | 0.00      |
| CodeGen-350M | 511          | 0.02     | 0.10     | 0.20      |

**Finding:** The relative ordering is preserved, but the absolute performance landscape changes sharply. GPT-2 Small and GPT-2 Medium remain exactly at zero, while CodeGen is only

marginally nonzero. For GPT-2 Small, one task did not yield a valid scored record under the custom export path, so pass@k is reported over 510 scored tasks; the other two models are scored over the full 511 tasks.

**Interpretation:** LiveCodeBench is therefore best read as a calibration result rather than a new positive benchmark win. It strengthens the paper’s negative claim that scale alone does not induce useful code generation in small general-language models, while also showing that a 350M code-pretrained model is still far from practically capable on a harder contamination-aware benchmark. The main positive CodeGen advantage is strongest on the classical Python benchmarks (HumanEval and MBPP), not on this harsher stress test.

#### 4.7 Within-Family Continued-Pretraining Control in CodeGen

To reduce the objection that GPT-2 versus CodeGen is “just a family difference,” we evaluate the CodeGen-350M NL→Multi→Mono checkpoint ladder described by Nijkamp et al. [12]. This comparison holds architecture and parameter count fixed while changing the added continued-pretraining corpus. We treat it as a strong within-family control, not as a pure distribution-only isolation, because later checkpoints also receive additional optimization steps and tokens.

Table 8: CodeGen-350M Continued-Pretraining Ladder

| Checkpoint    | HumanEval success % | MBPP success % |
|---------------|---------------------|----------------|
| CodeGen-NL    | 30.60               | 0.02           |
| CodeGen-Multi | 28.46               | 1.23           |
| CodeGen-Mono  | 37.76               | 2.76           |

**Finding:** Mono is strongest on both benchmarks, but the ladder behaves differently across them. On HumanEval, Mono significantly exceeds both NL (37.76% vs. 30.60%,  $p = 0.0001$ ) and Multi (37.76% vs. 28.46%,  $p = 0.0001$ ), while Multi trails NL slightly (28.46% vs. 30.60%,  $p = 0.0443$ ). On MBPP, the ladder becomes cleanly monotone: NL reaches 0.02%, Multi 1.23%, and Mono 2.76%, with significant improvements from NL to Multi ( $p = 0.0001$ ) and Multi to Mono ( $p = 0.0157$ ).

**Interpretation:** This is the strongest evidence in the paper against a pure family-comparison objection. Even within a single 350M architecture family, additional code- and Python-specific continued pretraining changes performance substantially. The HumanEval ladder is mixed because Multi underperforms NL, so we do not treat the ladder as a matched-compute law. The MBPP result is cleaner and supports the narrower claim that continued-pretraining plus curriculum effects materially improve small-model code generation within a single family.

As an auxiliary diagnostic, prompt perplexity on the ladder prompts is lowest for Mono on both HumanEval (4.87) and MBPP (4.46), intermediate for NL (4.93 and 4.64), and highest for Multi (5.54 and 5.23). Following CodeGen, we interpret these numbers only as prompt-checkpoint compatibility diagnostics, not as direct explanations of functional gains, so they remain secondary evidence rather than a main result.

#### 4.8 Layer Ablation Reveals Distinct Failure Profiles

To understand *why* scaling fails for GPT-2 but succeeds for CodeGen, we ablate each layer individually and measure the impact on code generation success.

Table 9: Signature Ablation Layers and Failure Modes

| Model        | Total Layers | Signature Layer(s)  | Residual Success         | Failure Signature (Ablated)                |
|--------------|--------------|---------------------|--------------------------|--------------------------------------------|
| GPT-2 Small  | 12           | Layer 2 (17%)       | 1.8%                     | 98.2% syntax                               |
| GPT-2 Medium | 24           | Layer 12 (50%)      | 0.0%                     | 76.1% syntax, 23.9% runtime                |
| CodeGen-350M | 20           | Layers 5, 7, 13, 18 | 11.7%, 4.6%, 29.5%, 0.1% | Mostly syntax; layer 13 adds 18.9% runtime |

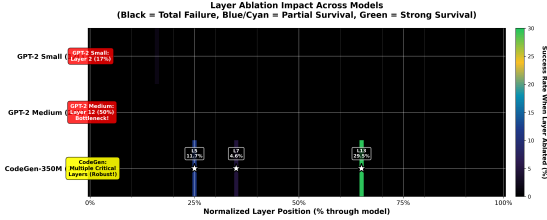


Figure 2: Success rates when each layer is ablated (zeroed out). Rows represent models, columns represent layer positions (normalized to 0-100% depth). Dark red indicates catastrophic failure (0% success), yellow indicates partial survival. GPT-2 Medium reaches 0% success under every ablation, with layer 12 standing out because it uniquely shifts failures toward runtime errors. CodeGen still shows severe sensitivity, but several layers retain partial survival, indicating broader residual robustness than either GPT-2 variant. Layer positions are normalized to enable cross-model comparison.

#### Key findings:

1. **GPT-2 Medium is uniformly brittle under full-layer ablation:** every single-layer ablation reduces success to 0%, so the model does not exhibit residual robustness anywhere in depth.
2. **Layer 12 is distinguished by failure mode, not residual success:** when layer 12 is ablated, GPT-2 Medium produces 23.9% runtime errors instead of collapsing

entirely to syntax errors. No other medium layer shows this pattern.

3. **CodeGen retains partial survival in several layers:** layers 5, 7, 13, and 18 all preserve nonzero success when ablated, with layer 13 preserving 29.5% success. This is still a harsh failure regime, but it is broader than either GPT-2 variant.
4. **Residual robustness and signature depth move together in this three-model snapshot:** GPT-2 Small’s only surviving layer appears early (17%), GPT-2 Medium’s distinctive runtime-shift layer appears at mid depth (50%), and CodeGen’s strongest residual layer appears later (65%). We treat this as descriptive evidence rather than a general law.

#### 4.9 Syntax Error Breakdown Reveals Different Learned Representations

To understand how pretraining shapes syntactic understanding, we categorize the types of syntax errors each model produces.

##### Key patterns:

1. **GPT-2 struggles with indentation:** Both GPT-2 models produce 22-29% indentation errors—far more than CodeGen (6.9%). Python’s whitespace sensitivity is

Table 10: Syntax Error Distribution by Type (% of all syntax errors)

| Model        | Ind. | Brack. | Quote | Kwd | Colon | Other |
|--------------|------|--------|-------|-----|-------|-------|
| GPT-2 Small  | 22.0 | 6.5    | 7.9   | 8.6 | 0.3   | 54.4  |
| GPT-2 Medium | 28.7 | 3.0    | 8.0   | 8.1 | 0.3   | 51.7  |
| CodeGen-350M | 6.9  | 21.8   | 5.1   | 2.1 | 4.4   | 59.7  |

not well-represented in general web text, leading to inadequate indentation models.

2. **Medium’s indentation errors increase with scale:** GPT-2 Medium (28.7%) produces more indentation errors than Small (22.0%), suggesting that scaling amplifies existing failure modes rather than correcting them.
3. **CodeGen fails on brackets:** CodeGen produces 21.8% bracket errors—7× more than GPT-2 Medium (3.0%). This likely reflects its training on bracket-heavy languages (Java: 35.5% of training data, C++: 8.2%) [12], where matching brackets is critical. The model overgeneralizes this constraint to Python, which uses brackets less frequently.
4. **Keyword and quote errors are similar:** All models produce 5-8% quote errors and 2-9% keyword errors, suggesting these are universally difficult syntactic constructs independent of pretraining.
5. **“Other” errors dominate:** 52–60% of syntax errors fall outside our six main categories, indicating long-tailed failure modes (invalid operators, malformed imports, incorrect statement structure) that no current model handles well.

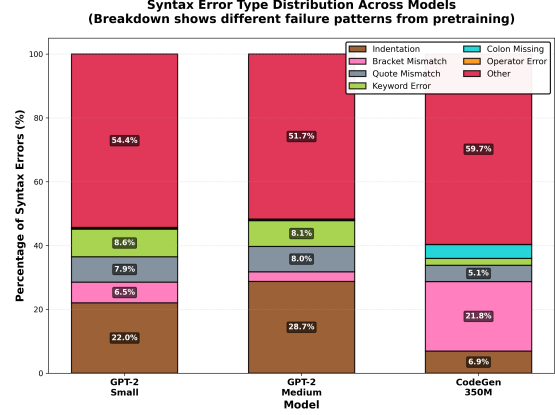


Figure 3: Distribution of syntax error types across models. GPT-2 models struggle primarily with indentation (22-29%), reflecting Python’s whitespace-sensitive syntax. CodeGen fails most often on brackets (21.8%), suggesting it learned bracket-heavy languages (Java, C++) during code pretraining. The “Other” category includes rare errors like invalid operators and malformed imports.

#### 4.10 Activation Analysis: A Compact Probe Signal at Layer 12

To understand the mechanistic basis of GPT-2 Medium’s distinctive layer-12 failure mode, we analyze which activation dimensions discriminate between successful and syntactically invalid generations.

**Methodology:** For 200 balanced samples (100 success, 100 syntax error), we mean-pool the hidden states  $h_{12} \in \mathbb{R}^{1024}$  from transformer block 12 and compute dimension-wise difference magnitudes:

$$\Delta_i = |\text{mean}(h_{12,i}^{\text{success}}) - \text{mean}(h_{12,i}^{\text{failure}})| \quad (6)$$

**Finding:** The strongest probe dimensions are

121, 346, 615, 820, and 169. A logistic regression probe on these five coordinates reaches 72.5% held-out accuracy, compared with 75.0% for the full-layer probe and 67.5% for the top-two-dimensional projection used in Figure 4.

Table 11: Top Discriminative Dimensions in Layer 12

| Dim                 | Mean (S) | Mean (F) | Diff. | % Sig.        |
|---------------------|----------|----------|-------|---------------|
| 121                 | -35.27   | -29.54   | 5.73  | 0.90%         |
| 346                 | 17.03    | 14.19    | 2.84  | 0.44%         |
| 615                 | 6.96     | 4.21     | 2.75  | 0.43%         |
| 820                 | -3.15    | -0.76    | 2.39  | 0.37%         |
| 169                 | -3.36    | -1.02    | 2.34  | 0.37%         |
| Remaining 1019 dims | —        | —        | —     | <b>97.49%</b> |

#### Key observations:

1. **A compact probe nearly matches the full layer:** The top-5 probe reaches 72.5% accuracy, which is 96.7% of the full-layer probe’s 75.0% accuracy.
2. **The signal is compact but not exclusive:** The same five dimensions account for only 2.51% of the total absolute mean shift, so much of the layer still moves between successful and failed generations even though a small probe captures most linear separability.
3. **Dimension 121 dominates the mean gap:** It shows a 5.73-unit difference between successful and failed generations, while the other top coordinates contribute smaller but consistent shifts.
4. **The 2D view is only partially separable:** The top-two-dimensional projection reaches 67.5% held-out accuracy with a class-mean separation of 6.39, so Figure 4

should be read as substantial but incomplete linear structure rather than a perfect boundary.

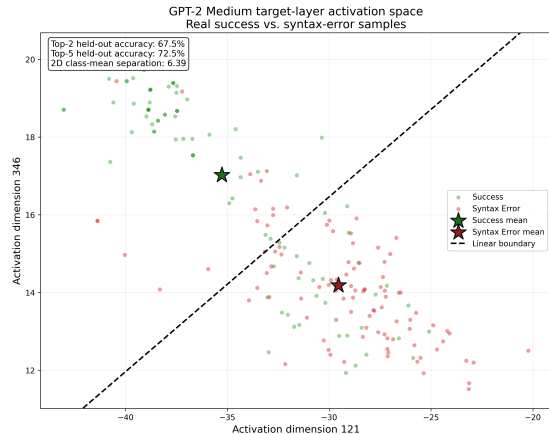


Figure 4: Layer-12 activations projected to the top two discriminative dimensions (121, 346) using real samples. Green points are successful generations and red points are syntax errors. The dashed line is a fitted linear boundary. A held-out probe on these two dimensions achieves 67.5% accuracy, while the top-five-dimensional probe reaches 72.5%, indicating substantial but incomplete separability.

#### 4.11 Cross-Model Comparison: Distinctive Ablation Profiles

To compare the three models more conservatively, we summarize the most informative ablation signature in each architecture rather than treating a single “critical depth” as a universal law.

**Finding:** Later signature layers coincide with stronger residual behavior in this three-model comparison, but we treat that pattern as descriptive rather than a scaling law. The clearest



Table 12: Signature Ablation Layers Across Models

| Model        | Signature Layer / Depth | Residual Success | Failure Signature           |
|--------------|-------------------------|------------------|-----------------------------|
| GPT-2 Small  | Layer 2 / 17%           | 1.8%             | 98.2% syntax                |
| GPT-2 Medium | Layer 12 / 50%          | 0.0%             | 76.1% syntax, 23.9% runtime |
| CodeGen      | Layer 13 / 65%          | 29.5%            | 51.6% syntax, 18.9% runtime |

contrast is qualitative: GPT-2 Small has only one ablation with any surviving success, GPT-2 Medium has none, and CodeGen retains useful behavior in several layers.

**Interpretation:** Early signature layers, such as GPT-2 Small’s layer 2, suggest that the model commits to correctness before enough context is available. GPT-2 Medium’s mid-depth layer 12 instead appears to assemble executable structure without preserving overall fault tolerance. CodeGen’s later layer 13 combines the strongest residual success with a mixed syntax/runtime failure pattern, consistent with more distributed code-relevant computation.

#### 4.12 Scaled Layer-12 Ablation: A Graded Intervention Pilot

Full zeroing establishes that layer 12 is special, but it does not reveal whether the layer degrades smoothly under weaker interventions. We therefore ran a smaller graded-ablation pilot on GPT-2 Medium layer 12 using 10 HumanEval problems and 5 samples per problem, scaling the layer output by  $\{0.75, 0.5, 0.25, 0.0\}$ .

**Finding:** The pilot is too small to treat the 0.75 condition as a stable improvement, but the broader pattern is informative. Partial weakening of layer 12 first drives the model toward near-total syntax collapse, whereas only full removal reproduces the runtime-error regime seen in the main hard-ablation study.

**Interpretation:** This suggests that layer 12

Table 13: Scaled Ablation Pilot for GPT-2 Medium Layer 12

| Scale    | Success % | Syntax % | Runtime % |
|----------|-----------|----------|-----------|
| Baseline | 6.0       | 94.0     | 0.0       |
| 0.75     | 8.0       | 92.0     | 0.0       |
| 0.50     | 2.0       | 98.0     | 0.0       |
| 0.25     | 0.0       | 100.0    | 0.0       |
| 0.00     | 0.0       | 72.0     | 28.0      |

is not merely a uniformly useful block. Instead, it appears to support a thresholded execution-relevant stage: modest attenuation disrupts syntactic coherence, while complete removal exposes a second regime in which partially executable but incorrect programs survive. That pattern is consistent with layer 12 acting as an assembly point for executable program structure rather than a generic source of accuracy.

#### 4.13 Constructive Steering and Specificity Controls

The probe results identify a compact readout, but a stronger mechanistic question is whether the same subspace can be used constructively during generation. We therefore steer GPT-2 Medium layer 12 with the top-5 contrast vector  $v = P_5(\mu_{\text{success}} - \mu_{\text{syntax}})$  and compare the result against matched sparse random controls that preserve sparsity and coefficient magnitudes but use different dimensions.

**Finding:** The best learned steering condition

Table 14: Layer-12 Steering and Matched-Control Pilot

| Condition                | Success % | Syntax % | Runtime % |
|--------------------------|-----------|----------|-----------|
| Baseline                 | 6.0       | 94.0     | 0.0       |
| Target $-2.0$            | 10.0      | 90.0     | 0.0       |
| Target $-1.0$            | 2.0       | 94.0     | 4.0       |
| Target $1.0$             | 4.0       | 96.0     | 0.0       |
| Target $2.0$             | 12.0      | 88.0     | 0.0       |
| Random mean ( $n = 20$ ) | 6.2       | 92.9     | 0.7       |
| Random max               | 14.0      | —        | —         |

( $\alpha = 2.0$ ) doubles subset success from 6.0% to 12.0%. Under an expanded set of 20 matched sparse random controls, the control mean remains near baseline at 6.2%, and the learned vector beats 19 of 20 controls, but one control reaches 14.0%. We therefore treat the constructive effect as real on this subset while treating the specificity evidence as suggestive rather than conclusive.

**Interpretation:** The sign pattern is not perfectly monotonic:  $\alpha = -2.0$  also improves success to 10.0%, while  $\alpha = -1.0$  introduces a small runtime regime. We therefore interpret the top-5 vector as a *locally useful intervention axis* rather than a globally linear notion of correctness. The strongest supported claim is that the identified subspace is not merely diagnostic, because targeted intervention along it can improve execution. The control comparison still points in the same direction, but with empirical  $p \leq 0.095$  it does not yet justify a stronger claim of clean subspace specificity.

#### 4.14 Scaled Layer-13 Ablation in CodeGen

To test whether the more robust code-specialized model also shows a graded signature, we ran the same 10-problem / 5-sample attenuation study

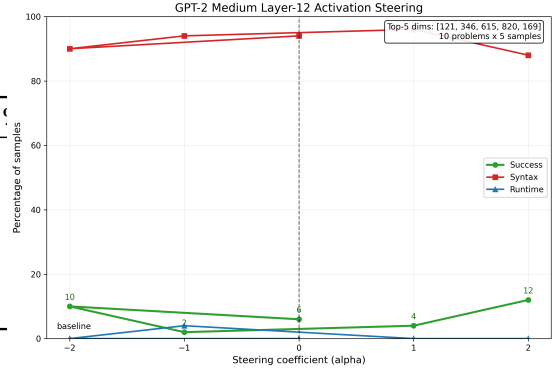


Figure 5: GPT-2 Medium layer-12 steering response on the 10-problem / 5-sample subset. The learned top-5 steering vector changes execution outcomes nontrivially, with the best positive setting ( $\alpha = 2.0$ ) doubling success from 6.0% to 12.0%. The non-monotonic curve indicates that the discovered subspace is intervention-sensitive but not a simple scalar “correctness knob.”

on CodeGen’s strongest residual layer (layer 13).

Table 15: Scaled Ablation Pilot for CodeGen Layer 13

| Scale    | Success % | Syntax % | Runtime % |
|----------|-----------|----------|-----------|
| Baseline | 40.0      | 60.0     | 0.0       |
| 0.75     | 28.0      | 72.0     | 0.0       |
| 0.50     | 30.0      | 70.0     | 0.0       |
| 0.25     | 2.0       | 98.0     | 0.0       |
| 0.00     | 24.0      | 52.0     | 24.0      |

**Finding:** Unlike GPT-2 Medium, CodeGen does not collapse immediately under partial attenuation. Layer 13 retains 28-30% success at scales 0.75 and 0.5, collapses to 2.0% success only at scale 0.25, and still preserves 24.0% success under full zeroing while shifting 24.0% of completions into runtime errors.

**Interpretation:** This is the paper’s clearest

graded cross-model contrast. GPT-2 Medium layer 12 acts like a brittle thresholded checkpoint, whereas CodeGen layer 13 behaves like a partially redundant execution stage. Even when layer 13 is fully removed, surrounding layers still support a meaningful fraction of executable completions. That is stronger evidence for distributed code-relevant computation than the full-zero ablation table alone.

## 5 Analysis and Discussion

### 5.1 Why Does Scaling Fail for GPT-2?

Our ablation results suggest that GPT-2 Medium’s failure is not a single isolated point of failure in the strict sense. Under full-layer zeroing, every block ablation reduces success to 0%. The new MBPP validation strengthens the broader empirical claim: the lack of improvement from GPT-2 Small to GPT-2 Medium is not confined to HumanEval. What distinguishes layer 12 is therefore not benchmark-specific residual success, but its HumanEval failure signature.

When layer 12 is removed, 23.9% of completions become runtime errors instead of collapsing entirely to syntax errors. No other GPT-2 Medium layer shows this pattern. This makes layer 12 the model’s clearest execution-relevant checkpoint: a place where partially coherent programs are still formed even though they no longer pass tests.

Relative to GPT-2 Small, scaling from 12 to 24 layers does not create broader residual robustness. GPT-2 Small at least retains 1.8% success when layer 2 is ablated, whereas GPT-2 Medium never retains success under any single-layer ablation.

A plausible interpretation is that general-language pretraining gives GPT-2 Medium enough mid-depth structure to produce executable code fragments, but not enough distributed code-specific computation to make that structure fault tolerant. Layer 12 appears to aggregate syntax into executable program form, while the surrounding layers do not preserve the computation under hard ablation.

The graded pilot sharpens this picture. When layer 12 is attenuated to 0.5 and 0.25 strength, syntax errors rise to 98.0% and 100.0% with no runtime errors. Only at 0.0 scale do runtime errors jump to 28.0%. We therefore read the runtime regime as a threshold effect rather than a smooth linear degradation: weakening layer 12 hurts syntax first, while completely removing it reveals partially executable but wrong code.

### 5.2 How Does CodeGen Achieve Robustness?

CodeGen’s superior performance does not mean it is immune to layer ablation. Many CodeGen ablations still reduce success to 0%, so the intervention is severe for all three models. The difference is that CodeGen shows a broader set of layers with residual nonzero success and more varied failure signatures than either GPT-2 variant.

**Layer 5 (25% depth):** retains 11.7% success under ablation, suggesting that early syntactic support is somewhat distributed rather than tied to a single block.

**Layer 7 (35% depth):** retains 4.6% success, again indicating partial resilience rather than total collapse.

**Layer 13 (65% depth):** preserves 29.5% success and 18.9% runtime errors, the strongest evidence that later code-relevant computation

can survive even after a full-block intervention.

The graded layer-13 pilot sharpens this picture. At scales 0.75 and 0.5, CodeGen still retains 28-30% success, so moderate attenuation reduces performance without destroying useful computation. Only at scale 0.25 does the layer collapse almost entirely into syntax errors (98.0%), while full zeroing reopens a mixed regime with 24.0% success and 24.0% runtime errors.

Taken together, these layers suggest a more distributed code-generation pipeline than in GPT-2. We interpret this as *partial* redundancy: enough to preserve useful behavior in a subset of ablations, but not enough to make the model robust to arbitrary block removal.

### 5.3 What the CodeGen Ladder Adds

The within-family CodeGen ladder tightens the paper’s central comparative claim. GPT-2 versus CodeGen alone leaves open the objection that different families, tokenizers, or implementation choices drive the entire result. The NL→Multi→Mono comparison weakens that objection because architecture and parameter count are held fixed while continued pretraining changes the data mix and curriculum.

The ladder does *not* support an overclean story. On HumanEval, Multi is slightly worse than NL, so the result is not a perfectly monotone progression. We therefore avoid claiming that pretraining distribution alone, under matched compute, is sufficient to explain every improvement. What the ladder does support is a narrower and stronger statement: once a fixed 350M architecture receives additional code-focused and then Python-focused continued pretraining, the best checkpoint (Mono) is clearly stronger than the earlier ones, and MBPP shows

a fully monotone NL<Multi<Mono trend.

An exploratory reduced ablation sweep across the ladder also suggests that the signature layer shifts with continued pretraining: NL’s strongest surviving layer is 11, Multi’s is 7, and Mono’s is 13. We treat these layer assignments cautiously because the ladder sweep is smaller than the main HumanEval ablation study, but they are consistent with the broader claim that continued pretraining changes not only accuracy but also where executable-code computation is supported.

### 5.4 Implications for Model Design

Our findings suggest three architectural principles for robust code generation:

**1. Distribute processing across depth:** Robust models should preserve some residual success or at least diverse failure signatures under harsh interventions, rather than collapsing uniformly at every layer.

**2. Track failure-mode transitions, not just success collapse:** Layer 12 in GPT-2 Medium is informative precisely because it changes the balance of syntax vs. runtime errors. The new scaled-ablation pilot suggests that this transition is thresholded: syntax collapses first, and the runtime regime appears only after complete removal.

**3. Code-specific continued pretraining is necessary:** General language pretraining (WebText) does not teach models how to organize code validation across depth. Code-specific pretraining (The Pile + GitHub) [5, 12] appears to produce broader residual robustness under the same intervention, and the within-family CodeGen ladder shows that later code- and Python-focused continued-pretraining stages can improve behavior even without changing archi-

texture or parameter count.

## 5.5 What the LiveCodeBench Result Changes

LiveCodeBench sharpens the scope of our claim. On HumanEval and MBPP, CodeGen’s advantage over GPT-2 is large and practically meaningful. On LiveCodeBench release\_v2, however, all three small models are near-zero, with CodeGen retaining only a marginal nonzero signal. That means the paper should not be framed as evidence that small code-pretrained models are broadly strong programmers. The better-supported statement is narrower: in the small-model regime, code-specific pretraining materially improves performance on classical Python synthesis benchmarks and changes the internal organization of code-relevant computation, but it does not close the gap to modern contamination-aware coding tasks.

This harsher benchmark is still useful scientifically. It makes the negative part of the paper stronger. GPT-2 Small and GPT-2 Medium remain indistinguishable at zero, so the failure of scale-alone generalization survives even when the benchmark changes substantially. At the same time, the compression of all three models toward zero warns against overgeneralizing from HumanEval or MBPP alone.

## 5.6 Why a Small Probe Still Works

The five-dimensional probe is informative because it captures most of the linear separability available in this layer: 72.5% held-out accuracy versus 75.0% for the full 1024-dimensional linear probe. In that operational sense, a small coordinate set provides a compact readout of success vs. syntax failure.

However, this does *not* mean that only five neurons implement correctness. The top five dimensions account for just 2.51% of the total absolute mean shift across the layer, indicating that many weaker coordinates also move between successful and failed generations.

The constructive steering results refine this interpretation. The best positive steering setting doubles success from 6.0% to 12.0%. Against 20 matched sparse random controls, this intervention exceeds the control mean and beats 19 controls, but it is not the single best perturbation in the set. At the same time, the response curve is not monotonic: strong negative steering also improves success, while weaker negative steering produces a small runtime regime. That pattern is more consistent with a useful local intervention subspace than with a single signed “correctness neuron” explanation.

We therefore interpret these dimensions as especially stable readout axes rather than a complete mechanistic decomposition. They are useful for monitoring and future causal interventions [11], but the broader representation remains distributed.

## 5.7 Limitations and Future Work

Our study has several limitations that future work should address:

**1. Benchmark scope is broader but still limited:** We now evaluate on HumanEval, HumanEval+, MBPP, MBPP+, and contamination-aware LiveCodeBench [2, 1, 10, 7], which is stronger than the earlier two-benchmark version of the paper, but all settings still emphasize short-form code-generation tasks rather than long-horizon repository-level software engineering.

**2. Pretrained models only:** We do not

evaluate fine-tuned models. While our goal was to isolate base architecture effects, fine-tuning could potentially repair GPT-2 Medium’s bottleneck by redistributing computation across layers.

**3. The CodeGen ladder is controlled but not matched-compute:** NL, Multi, and Mono hold architecture and parameter count fixed, but they also differ in additional tokens, optimization steps, and curriculum. The ladder therefore supports a strong within-family continued-pretraining claim, not a perfectly factorized causal isolation of data distribution alone.

**4. Graded interventions remain limited:** We now include scaled-ablation pilots for GPT-2 Medium layer 12 and CodeGen layer 13, but both cover only one layer at a time on 10-problem / 5-sample subsets. Broader graded sweeps remain necessary before claiming a general law of thresholded behavior.

**5. Limited semantic analysis:** We identify which layers validate syntax but do not deeply analyze which layers perform semantic reasoning (variable scoping, type consistency, algorithm correctness). Future work should extend our methods to semantic error detection.

**6. Constructive intervention evidence is still small-scale:** We now observe positive rescue from learned steering on a subset, and the learned vector beats 19 of 20 matched sparse random controls, but one control does better and the empirical control test remains only suggestive ( $p \leq 0.095$ ). Larger control sets and cross-benchmark steering experiments are needed before treating the subspace as a robust causal handle.

Despite these limitations, our core finding is that code models differ not only in accuracy but in their layerwise failure profiles under intervention, providing actionable insight for model design and evaluation.

## 6 Conclusion

We have shown that the conventional wisdom “bigger models are better” breaks down for code generation, with GPT-2 Medium (355M parameters) failing to improve meaningfully over GPT-2 Small (124M parameters) despite  $2.9\times$  more parameters.

Problem-level bootstrap analysis makes this conclusion more precise: GPT-2 Small and GPT-2 Medium are statistically indistinguishable on HumanEval in our setting, whereas both remain more than 30 percentage points behind CodeGen-350M with highly significant pairwise differences.

The same claim transfers to full-coverage MBPP. Across 257 tasks with 20 samples each, GPT-2 Small and GPT-2 Medium both achieve 0.0% success, whereas CodeGen-350M reaches 7.39% success and 22.96% pass@5. The pairwise MBPP success-rate gaps against CodeGen remain highly significant ( $p = 0.0001$ ), while the GPT-2 Small vs. GPT-2 Medium difference vanishes entirely ( $p = 1.0$ ).

A stricter HumanEval+ re-evaluation preserves the same ordering. On the first five saved samples per task, GPT-2 Small and GPT-2 Medium both remain at 0.0% pass@1, whereas CodeGen-350M retains 2.1% pass@1 after the augmented hidden tests.

The same conclusion survives a separate strict MBPP+ evaluation with EvalPlus. On 378 tasks scored under the stronger Base+Extra metric, GPT-2 Small and GPT-2 Medium both remain at 0.0% pass@1 and pass@10, whereas CodeGen-350M retains 1.1% pass@1 and 6.4% pass@10.

The contamination-aware LiveCodeBench result narrows and strengthens the paper at the same time. GPT-2 Small and GPT-2 Medium remain at 0.0% pass@1/pass@5/pass@10, while

CodeGen-350M remains only marginally above zero (0.02%/0.10%/0.20%). So the paper’s strongest positive comparative result remains concentrated on HumanEval and MBPP, but the negative conclusion that scale alone is insufficient survives even under a much harder modern benchmark.

The within-family CodeGen ladder sharpens the paper further. At fixed 350M scale, CodeGen-Mono is strongest on both HumanEval and MBPP, while the MBPP ladder is cleanly monotone from NL to Multi to Mono. Because HumanEval is mixed and the checkpoints receive additional compute and curriculum, we interpret this not as a pure data-only law but as strong within-family evidence that later code- and Python-focused continued pretraining materially improves small-model code generation.

Under full-layer ablation, GPT-2 Medium never preserves success; instead, layer 12 stands out because its removal uniquely shifts 23.9% of failures into runtime errors while every other layer collapses almost entirely to syntax errors.

A graded layer-12 pilot strengthens this interpretation. Partial attenuation first pushes GPT-2 Medium toward complete syntax collapse, while only full zeroing reopens a 28.0% runtime-error regime. This suggests that layer 12 marks a thresholded execution-relevant stage rather than a smoothly redundant computation.

In contrast, CodeGen-350M [12] still shows severe sensitivity to hard ablation, but it exhibits broader residual robustness than either GPT-2 variant. Four layers retain nonzero success, and the graded layer-13 pilot preserves 28-30% success under partial attenuation and 24.0% success even under full zeroing, indicating that code-relevant computation is less concentrated than in GPT-2.

Our activation analysis shows that a compact

probe on five layer-12 dimensions (121, 346, 615, 820, 169) reaches 72.5% held-out accuracy, close to the 75.0% achieved by a full-layer linear probe. More importantly, constructive steering along this same sparse direction doubles subset success from 6.0% to 12.0%. The expanded random-control comparison is favorable but not decisive, so the safest conclusion is that this direction is a usable local control subspace rather than a purely descriptive probe.

These findings challenge the scaling-centric paradigm in neural code generation [8, 18]. Parameter count alone does not determine capability. Across HumanEval, HumanEval+, MBPP, MBPP+, and a harsher contamination-aware LiveCodeBench stress test, the central predictor of useful code behavior in this small-model regime is not scale within the GPT-2 family, but whether pretraining has already distributed code-relevant computation across depth. The within-family CodeGen ladder makes this point harder to dismiss: even with architecture and parameter count held fixed, later continued-pretraining stages can materially change both performance and the layerwise organization of executable-code behavior. For practitioners, the main design lessons are to distribute code-relevant processing across depth, analyze failure-mode shifts under intervention, and use code-specific pretraining to build residual robustness.

Future work should extend our mechanistic analysis to fine-tuned models, multilingual code generation, and semantic error detection, with the ultimate goal of reverse-engineering the complete algorithms by which neural networks generate syntactically and semantically correct programs.

## References

- [1] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, and Charles Sutton. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*, 2021.
- [2] Mark Chen, Jerry Tworek, Heyang Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicolas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgren Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N Carr, Jan Leike, Josh Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.
- [3] Kevin Clark, Urvashi Khandelwal, Omer Levy, and Christopher D Manning. What does bert look at? an analysis of bert’s attention. *arXiv preprint arXiv:1906.04341*, 2019.
- [4] Arthur Conmy, Augustine N Mavor-Parker, Aengus Lynch, Stefan Heimersheim, and Adrià Garriga-Alonso. Towards automated circuit discovery for mechanistic interpretability. *Advances in Neural Information Processing Systems*, 36, 2023.
- [5] Leo Gao, Stella Biderman, Sid Black, Laurence Golding, Travis Hoppe, Charles Foster, Jason Phang, Horace He, Anish Thite, Noa Nabeshima, et al. The pile: An 800gb dataset of diverse text for language modeling. *arXiv preprint arXiv:2101.00027*, 2021.
- [6] Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, et al. Training compute-optimal large language models. *arXiv preprint arXiv:2203.15556*, 2022.
- [7] Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. Livecodebench: Holistic and contamination free evaluation of large language models for code. *arXiv preprint arXiv:2403.07974*, 2024.
- [8] Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*, 2020.
- [9] Yujia Li, David Choi, Junyoung Chung, Nate Kushman, Julian Schrittwieser, Rémi



- Leblond, Tom Eccles, James Keeling, Felix Gimeno, Agustin Dal Lago, et al. Competition-level code generation with alphacode. *Science*, 378(6624):1092–1097, 2022.
- [10] Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. Is your code generated by chatgpt really correct? rigorous evaluation of large language models for code generation. *arXiv preprint arXiv:2305.01210*, 2023.
- [11] Kevin Meng, David Bau, Alex Andonian, and Yonatan Belinkov. Locating and editing factual associations in gpt. *Advances in Neural Information Processing Systems*, 35:17359–17372, 2022.
- [12] Erik Nijkamp, Bo Pang, Hiroaki Hayashi, Lifu Tu, Huan Wang, Yingbo Zhou, Silvio Savarese, and Caiming Xiong. Codegen: An open large language model for code with multi-turn program synthesis. *arXiv preprint arXiv:2203.13474*, 2022.
- [13] Catherine Olsson, Nelson Elhage, Neel Nanda, Nicholas Joseph, Nova DasSarma, Tom Henighan, Ben Mann, Amanda Askell, Yuntao Bai, Anna Chen, Tom Conerly, Dawn Drain, Deep Ganguli, Zac Hatfield-Dodds, Danny Hernandez, Scott Johnston, Andy Jones, Jackson Kernion, Liane Lovitt, Kamal Ndousse, Dario Amodei, Tom Brown, Jack Clark, Jared Kaplan, Sam McCandlish, and Chris Olah. In-context learning and induction heads. *arXiv preprint arXiv:2209.11895*, 2022.
- [14] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, Ilya Sutskever, et al. Language models are unsupervised multitask learners. *OpenAI blog*, 1(8):9, 2019.
- [15] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in Neural Information Processing Systems*, 30, 2017.
- [16] Jesse Vig, Sebastian Gehrmann, Yonatan Belinkov, Sharon Qian, Daniel Nevo, Yaron Singer, and Stuart Shieber. Investigating gender bias in language models using causal mediation analysis. *Advances in Neural Information Processing Systems*, 33:12388–12401, 2020.
- [17] Elena Voita, David Talbot, Fedor Moiseev, Rico Sennrich, and Ivan Titov. Analyzing multi-head self-attention: Specialized heads do the heavy lifting, the rest can be pruned. *arXiv preprint arXiv:1905.09418*, 2019.
- [18] Jason Wei, Yi Tay, Rishi Bommasani, Colin Raffel, Barret Zoph, Sebastian Borgeaud, Dani Yogatama, Maarten Bosma, Denny Zhou, Donald Metzler, et al. Emergent abilities of large language models. *Transactions on Machine Learning Research*, 2022.